eXtended Reality Coding

*Note: Sub-titles are not captured in Xplore and should not be used

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

4th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

5th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

6th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—This work presents a real-time volumetric streaming architecture based on a 3D-to-2D projection strategy designed to exploit conventional H.265/HEVC video compression mechanisms. Instead of directly encoding Point Cloud Data (PCD) in the spatial domain, the system converts volumetric information into synchronized multi-view 2D projections organized into a structured atlas representation. The generated super-frame is encoded as a native YUV/I420 stream and transmitted through a low-latency UDP pipeline.**

The proposed architecture includes both an Encoder and a Decoder subsystem. The Encoder transforms raw point cloud data into Geometry, Texture, and Occupancy maps through orthogonal projections and G-buffer extraction. The Decoder reconstructs the volumetric scene by applying inverse projection techniques and synchronized attribute restoration.

The system also integrates a bidirectional feedback mechanism allowing interactive manipulation of the volumetric scene through real-time user commands such as yaw, pitch, zoom, and temporal skipping. Experimental observations demonstrate that the proposed MPEG-emulation strategy enables efficient real-time streaming while preserving geometric fidelity and maintaining stable 30 FPS performance.

***Index Terms*———**

I. Introduction(Sort of Abstract)

This experiment employs a technique designed to capitalize on established video encoding mechanisms. Rather than compressing data directly in the 3D domain, it projects the 3D point cloud onto multiple 2D planes, converting spatial and visual information into a sequence of images that can be processed as a standard video stream.

Visual information, captured by a single camera,(or a multi-camera system) is initially represented as Point Cloud Data (PCD).To prevent the severe degradation of spatial structure caused by direct 3D compression, the encoder applies an intermediate 3D-to-2D transformation. This step effectively translates the 3D representation into a format suitable for conventional video compression. Once the data are converted into a standard video format, the encoder proceeds with the actual compression.

Identify applicable funding agency here. If none, delete this.

More specific:

- This experiment employs a technique designed to capitalize on established 2D video encoding mechanisms for volumetric data streaming. Instead of compressing the 3D Point cloud Data directly, the system projects the data onto a structured 2D layout (G-Buffer) representing multiple viewpoints.

- This transformation translates the 3D representation into a format suitable for high-efficiency video codecs, specifically H.265 (HEVC). By acting as an "MPEG emulator," the encoder organizes geometry and texture data into a native YUV/I420 format, allowing the system to leverage hardware-accelerated video compression for real-time volumetric streaming.

II. Data Acquisition and Source

The system's core processing pipeline is fed by a continuous stream of volumetric data. These data originate from the 8i Voxelized Full Bodies (8iVFB) dataset, specifically utilizing the "Loot" sequence, which is a widely recognized benchmark in the field of 3D point cloud compression and research.

The input consists of raw binary Point Cloud Data (PCD). In the context of the system's architecture, the volumetric frames are transmitted from a dedicated upstream station (Encoder A) and received via a TCP connection. Rather than processing isolated files, the system ingest these binary packets to create a continuous volumetric flux. This stream serves as the primary input for the subsequent conversion stages, where the 3D information is dynamically projected and transformed into a standard video format.

III. From Binary Stream to Spatial Data

The first operation performed by the encoder is the ingestion of the raw binary stream arriving via the TCP connection. This stage represents a critical process of data interpretation and spatial reconstruction rather than a simple decoding, as the system must transform raw network packets into a coherent volumetric structure. It is a critical process of data interpretation and spatial reconstruction.

Each incoming packet starts with a fixed 32-byte binary header. This header acts as the control logic for the stream, containing essential metadata such as the frame ID, high-precision timestamps, size information and more. Using the size information extracted from the header, the system isolates the binary payload. This raw data chunk contains the coordinates and attributes of the point cloud. Once the payload is isolated, the system converts this raw binary chunk into a manipulable 3D object using the Open3D library.

IV. 3D-to-2D Projection

Once the volumetric data is reconstructed, the system enters the "Virtual Stage" phase. The raw points are transformed starting from unorganized 3D point cloud data into a structured multi-layered 2D representation compatible with standard video codecs. This stage acts as a high-speed bridge between spatial geometry and cinematic frames.

Unlike traditional voxelization or segmentation methods, this system utilizes a *Full-Surface Multi-view Orthogonal Projection* to map volumetric data onto a set of synchronized 2D image planes. A defining feature of this architecture is its interactivity: the projection is not static, but dynamically adapts its orientation (Yaw and Pitch) and scale (Zoom) in response to real-time user input. The engine projects the entire point cloud onto the six faces of an adaptive bounding volume simultaneously (Front, Back, Right, Left, Top, Bottom). This generates a comprehensive G-Buffer—a multi-view representation.

A. Object-Centric Normalization and View-Adaptive Domain Definition

The transition from raw sensor data to a video-ready representation requires the establishment of an *Object-Centric Reference Frame*. Raw volumetric data are typically defined in coordinates relative to the sensor; however, this is unsuitable for temporal compression because any global movement would force the video encoder to compensate for massive global translations rather than local surface changes.

1) Translation Invariance and Barycenter Calculation: To achieve *Translation Invariance*, the "Virtual Director" must first center the actor on the stage. This minimizes the "orbital" movement of points, which is critical for the motion estimation algorithms of the video codec.

Given a raw point cloud $\mathcal{P} = p_i = (x_i, y_i, z_i)$ containing N points, the barycenter $\mathbf{C}$ is calculated as the mean of all spatial coordinates:

$$\mathbf{C} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{p}_i = \left(\frac{1}{N}\sum_{i=1}^{N}x_i, \frac{1}{N}\sum_{i=1}^{N}y_i, \frac{1}{N}\sum_{i=1}^{N}z_i\right)$$

Each point is then translated to the origin: $\mathbf{p}'_i = \mathbf{p}_i - \mathbf{C}$. By centering the object, the system ensures that subsequent rotations (Yaw and Pitch) and scaling (Zoom) occur around the geometric center, significantly reducing pixel displacement between consecutive frames(the "orbital" movement of points).

2) Interactive Pose Adaptation and Auto-Scaling: Once the point cloud is centered at the origin, the "Virtual Director" applies dynamic transformations based on real-time feedback from the client. In this architecture, the normalization process is not a static pre-processing step but a responsive engine that updates the object's pose for every frame:

- **Dynamic Rotation (Yaw and Pitch):**
  The system applies rotation matrices around the Y (Yaw) and X (Pitch) axes. By performing these rotations on the centered coordinates $\mathbf{p}'_i$, we eliminate the "orbital" displacement, ensuring that even a 180° turn keeps the actor perfectly framed within the 2D patches.
- **Intelligent Auto-Scaling:**
  To ensure the object occupies the maximum possible area of the *Width* $\times$ *Height* canvas without clipping, the system calculates the maximum radius of the point cloud from the center. It then derives a *final_scale* factor that adapts the object's size to the virtual camera's FOV and distance.
- **The Z-Push (Camera Distance):**
  Finally, a translation along the Z-axis (`camera_dist`) is applied. This "pushes" the object to a specific distance from the virtual camera rig (typically 1200 units), establishing the baseline for the subsequent depth-mapping and G-Buffer generation.

3) Dual-Domain Adaptive Bounding Volume: Once centered, the system defines the projection volume that establishes the transition from the raw sensor coordinate space to the normalized 2D projection domain. While standard processing typically relies on a simple *Axis-Aligned Bounding Box (AABB)* to identify the spatial extents of the object, our system implements a *Dual-Domain Adaptive Bounding Volume*. This approach ensures that the volumetric data is perfectly fitted to the target video resolution, Width $\times$ Height, while optimizing pixel utilization on the 2D canvas.

To define this volume, the system calculates the physical extents of the object (E_x, E_y, E_z) and applies a global padding factor ($\gamma = 1.1$, providing a "10%" margin).margin). This adaptive margin is essential for the interactive nature of the system, as it prevents points from clipping against the edges of the 2D patches during real-time Yaw and Pitch rotations. To maintain absolute geometric fidelity, the system derives a *Rigorous Unique Scale* S_{global} based on the maximum spatial requirement relative to the target resolution:

$$S_{global} = \max\left(\frac{E_x}{W}, \frac{E_y}{H}, \frac{E_z}{W}\right) \cdot \gamma$$

where W and H are the pixel dimensions of the individual views (e.g., 640×480). By applying this uniform scale, the system ensures *Proportional Mapping*, where each pixel in the 2D domain represents a constant metric distance in 3D space across all six cardinal views. This resulting volume is then projected into individual 2D patches using two synchronized strategies: The resulting volume is then projected into individual 2D images, formally known as Patches. To fit these into a standard 640×480 video resolution without severe distortion, the system adopts two different strategies:

- **Lateral Views (Front, Back, Right, Left):** These views utilize a rectangular domain that respects the standard video aspect ratio $(4:3)$. By using the S_{global} factor, the system maximizes the occupied area within the frame while ensuring that the object's proportions are preserved. This logic minimizes padding (empty space) and provides a coherent surface for the motion estimation algorithms of the codec.
- **Top and Bottom Views (Zenith and Nadir):** To preserve the object's planar footprint, these views maintain a proportional spatial mapping centered within the Width $\times$ Height patch. Rather than stretching the projection to fill the rectangular frame, the system utilizes a uniform global scale that naturally results in pillarboxing—leaving empty black bands at the lateral edges of the pixel width. This "padded" approach ensures a $1:1$ metric consistency with the lateral views, allowing the client to reconstruct the volumetric data without the need for non-linear aspect ratio corrections.

At the conclusion of this operation, the system generates multiple typologies of data for each 2D projection. Although these represent distinct physical attributes, they are organized into synchronized Patches, which serve as the fundamental building blocks for the subsequent G-Buffer extraction and Atlas assembly.

B. Multi-view Orthogonal Projection Engine

Once the spatial domain is established, the system functions as a multi-camera rig, projecting the 3D point cloud onto the six faces of the adaptive bounding volume. This process generates six synchronized 2D image planes, known as Patches. To preserve the exact metric scale of the object and avoid geometric distortions such as perspective foreshortening, the system utilizes an *Orthogonal Projection* model.

1) Mathematical Framework: Using homogeneous coordinates, a centered 3D point $\mathbf{p}$ can be represented as $\mathbf{p} = [x, y, z, 1]^T$. The projection onto a principal plane (e.g., the Front view on the xy-plane) is defined by the projection matrix $\mathbf{P}$:

$$\mathbf{P} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{1}$$

Applying this transformation yields the projected point $\mathbf{p}' = \mathbf{P} \cdot \mathbf{p} = [x, y, 0, 1]^T$. While this operation collapses one spatial dimension to create a 2D map, the discarded coordinate—representing the orthogonal depth d—is preserved to populate the Geometry Map.

This transformation is generalized for all six cardinal directions by permuting the axes. In the system's implementation, the normalized coordinates (n_x, n_y, n_z) are mapped to the UV image plane according to the target face:

- Front/Back: Project onto the XY plane; Z is stored as depth d.
- Right/Left: Project onto the ZY plane; X is stored as depth d.
- Top/Bottom: Project onto the XZ plane; Y is stored as depth d.

To handle the computational load of processing millions of points per frame across six views, and to ensure real-time performance (30 FPS) the engine execute the mathematical mapping is implemented within a highly optimized framework

C. Visibility Determination and Z-buffering

A volumetric point cloud is inherently porous; multiple points may project onto the exact same (u, v) pixel coordinate. To resolve these self-occlusions and extract only the "visible skin" of the object, the engine implements a strict Z-buffering mechanism. To resolve these self-occlusions, a visibility determination algorithm is mandatory.

1) Depth Resolution Logic: To resolve self-occlusions, the engine implements a visibility determination algorithm. Mathematically, for each pixel (u, v), the depth buffer $D(u, v)$ preserves only the point closest to the projection plane, which effectively represents the outermost surface (or 'visible skin') of the volumetric object:

$$D(u, v) = \min d_i \mid \text{proj}(\mathbf{p}_i) = (u, v) \tag{2}$$

A local depth buffer is initialized to a baseline value for each face, allowing the system to resolve occlusions for all six views concurrently using a parallel loop:

$$\text{If } d > \text{DepthBuffer}[face, v, u] \implies \Big\{ \text{DepthBuffer}[face, v, u] \leftarrow d \text{ Upda} \tag{3}$$

By resolving visibility at the projection stage, the engine produces "clean" patches where each pixel corresponds to a unique point in space. This step is critical for encoding efficiency. By rendering only the solid outer shell, the system removes internal "noise" that would otherwise create erratic pixel variations. This simplified, smooth surface maximizes the efficiency of the temporal prediction algorithms used in the H.265/HEVC codec, as it minimizes pixel displacement and entropy between consecutive frames.

D. Feature Extraction and Multi-layered G-Buffer

Once the visibility is resolved through Z-buffering, the engine extracts the physical and photometric attributes of the visible surface. In this architecture, the pixel coordinate (u, v) acts as a common spatial anchor: while the coordinates identify the projection of a specific 3D point, the values stored

within that position vary across three distinct layers to describe different attributes of the volumetric data.

1) The Three Information Layers: To ensure a complete reconstruction, the system populates a multi-layered G-Buffer where each layer serves a specific role in the 3D-to-2D transformation:

- Geometry Map (Spatial Structure) This layer stores the quantized depth information. To optimize the signal for 8-bit video compression, the continuous depth d is mapped to the Luminance (Y) channel. Since physical surfaces are locally smooth, this mapping creates high spatial correlation, which is extremely efficient for the H.265 motion estimation algorithms.

$$Y(u,v) = 255 - \left\lfloor \frac{d + (S_{max}/2)}{S_{max}} \cdot 255 \right\rfloor \tag{4}$$

  Theoretical implication: Since physical surfaces tend to be locally smooth, encoding depth as luminance leverages high spatial correlation. This allows conventional video codecs to compress the geometric structure with extremely high efficiency.
- Texture Map (Visual Appearance) This is a 3-channel layer characterizing the photometric attributes (color) of the points. To minimize latency, the system performs an in-line RGB to YUV (BT.601) conversion during the projection phase, populating the Y, U, and V components directly.

$$Y = 0.299R + 0.587G + 0.114B$$

$$U = -0.1687R - 0.3313G + 0.500B + 128$$

$$V = 0.500R - 0.4187G - 0.0813B + 128$$

  Theoretical implication: While geometry is generally smooth, attribute data exhibits high-frequency spatial variations due to sharp color gradients and complex textures.
- Occupancy Map (Signaling Filter) The Occupancy Map is a binary signaling matrix identifying active pixels:255 for "object" and 0 for "background. This map acts as a vital mask for the decoder, preventing the reconstruction of "ghost" points in empty space and allowing the encoder to focus the bitrate budget only on the object's surface. The map guide the decoder to distinguish between the object's surface and the empty background:

$$O(u,v) = \begin{cases} 255 & \text{if } \exists \mathbf{p}_i \text{ at } (u,v) \\ 0 & \text{otherwise} \end{cases} \tag{5}$$

  Theoretical implication: This map is vital for coding efficiency, preventing the video encoder from wasting computational resources on empty background space.

2) Planar Component Mapping (YUV I420): The true innovation in the transition from spatial data to a video bitstream lies in the *Planar Component Mapping*. Instead of treating the maps as standard images, the engine organizes them according to the YUV 4:2:0 (I420) memory hierarchy. This separation allows the system to prioritize structural precision over color density.

- The Y (Luminance) Plane: The Luminance channel represents the "brightness" or grayscale intensity of the scene. Acts as the primary data carrier. It contains the full-resolution information for all three atlases (Geometry, Texture, and Occupancy). By storing depth and masks as luminance, the system ensures that these critical spatial descriptors are preserved with maximum precision.
- The U and V (Chrominance) Planes: These planes are utilized exclusively for the Texture Map, because carry the color information: U for blue-difference and V for red-difference . For the Geometry and Occupancy layers, these components are populated with a neutral constant (128). This strategy "blanks" the color information for non-photometric data, focusing the encoder's computational power on the structural details.

Theoretical Implication By decoupling the Luminance (where the 3D structure lives) from the Chrominance (where the aesthetic color lives), the system emulates a professional broadcast signal. This allows the H.265 codec to apply different compression levels: it maintains high "sharpness" for the Geometry to keep the 3D shape intact, while safely compressing the Texture colors to save significant bandwidth without a perceived loss in quality for the end-user.

E. Atlas Generation and Super-Frame Assembly

Once the multi-layered patches are generated, they must be organized into a high-resolution spatial arrangement. In this architecture, the individual patches for each typology (Geometry, Texture, and Occupancy) are aggregated into a single 2D canvas known as the Atlas. To maximize compatibility with hardware-accelerated video codecs, the system does not simply concatenate images, but rather constructs a native I420 (YUV 4:2:0) planar buffer.

1) Atlas Deterministic Cross-Shaped Layout: A critical design choice in this architecture is the implementation of a Fixed-Layout Mosaic strategy. While traditional volumetric standards, such as MPEG V-PCC, often utilize dynamic patch-packing to minimize empty space, our system strictly fixes the spatial location of each cardinal view within a 3×4 grid. In the encoder implementation, this is achieved through a deterministic coordinate mapping—ensuring that each of the six faces occupies a constant region within the Atlas.This stable, deterministic layout, is fundamental to optimize spatial organization, achieving high compression ratios and maintaining real-time fluidity The views are arranged in a "cross-shaped" layout according to the following cardinal mapping:

- **Row 1:** Empty, Top view, Empty, Empty
- **Row 2:** Front view, Right view, Back view, Left view

[b]0.3

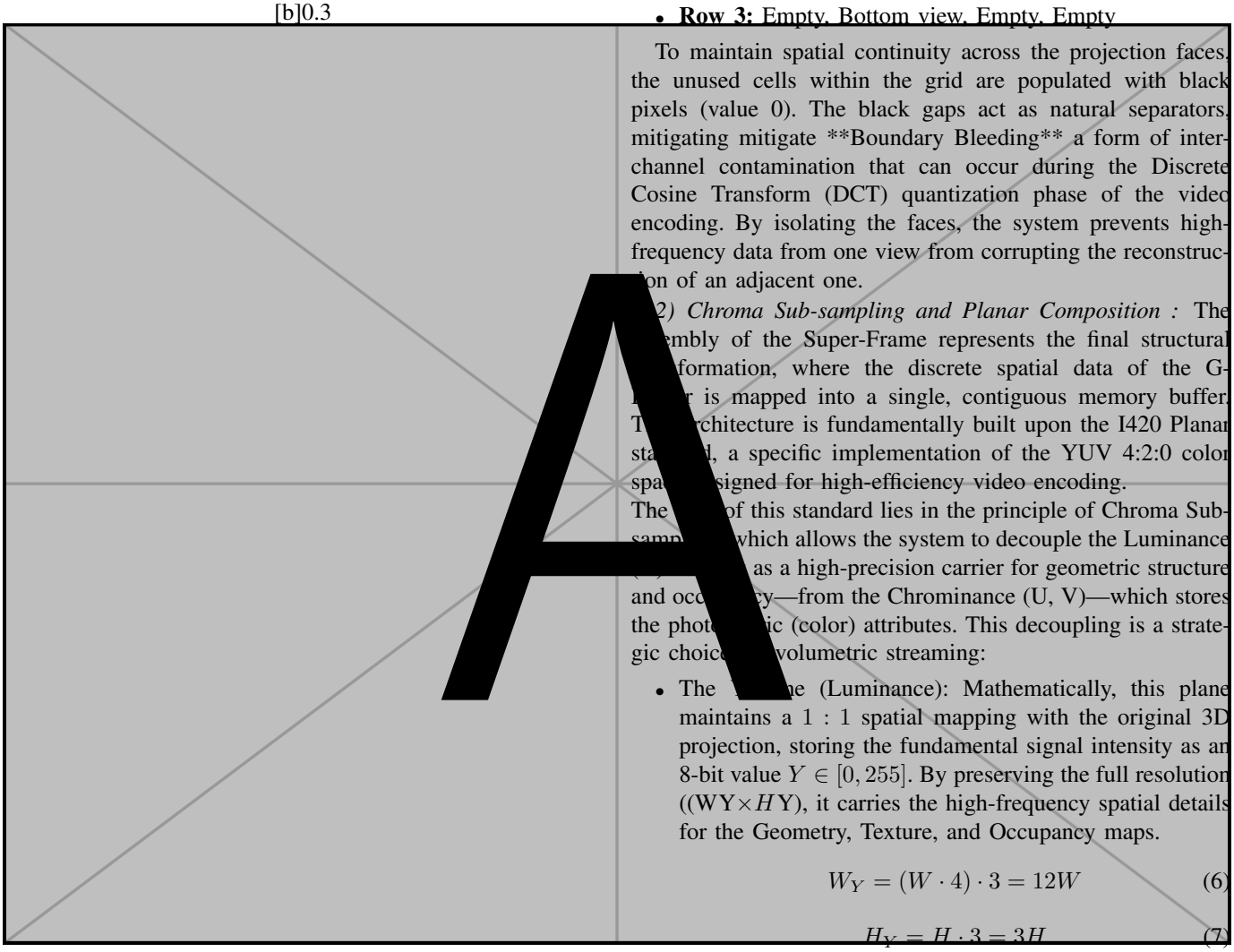
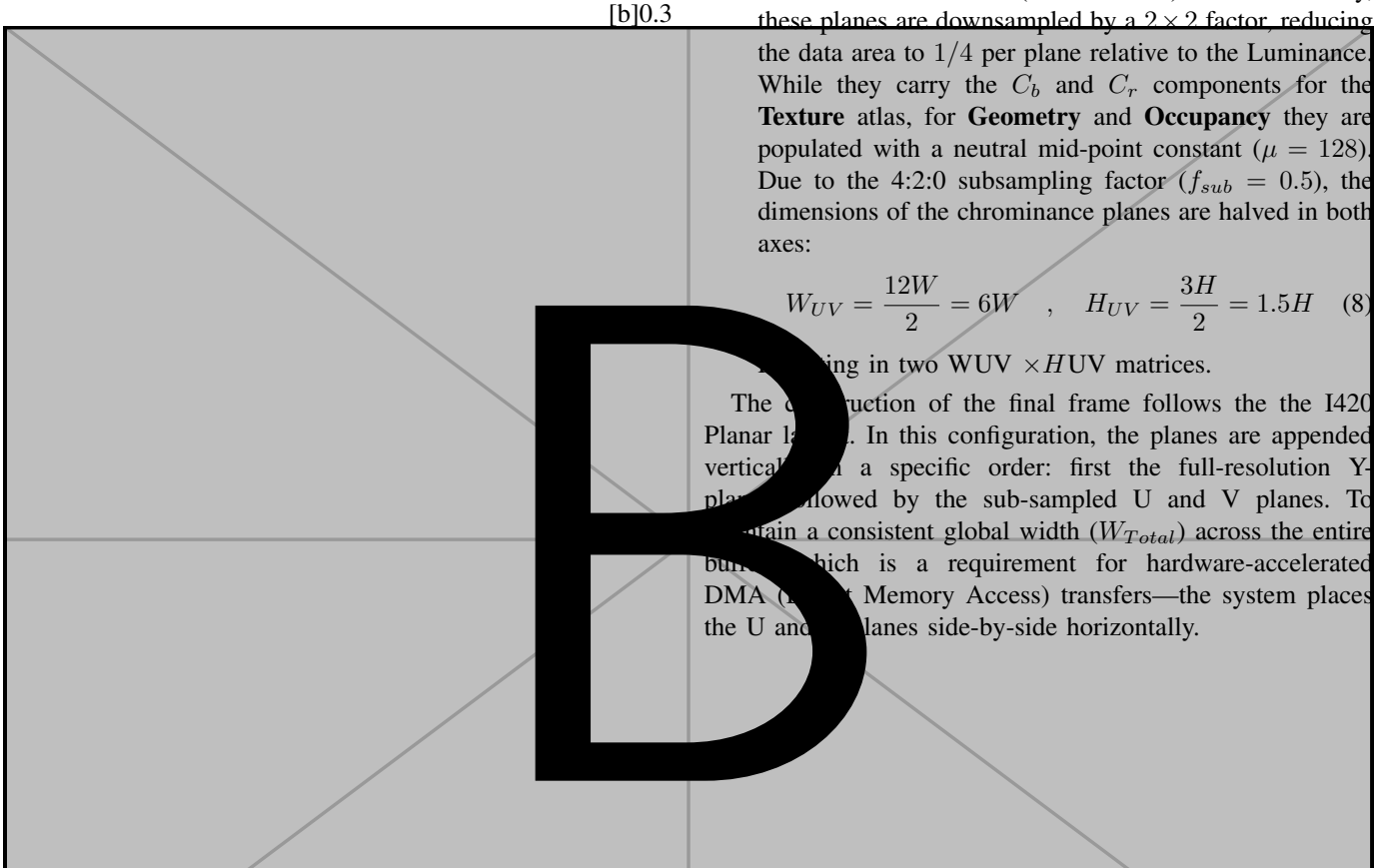


Fig. 1. Front Patch

[b]0.3



- **Row 3:** Empty, Bottom view, Empty, Empty

To maintain spatial continuity across the projection faces, the unused cells within the grid are populated with black pixels (value 0). The black gaps act as natural separators, mitigating mitigate **Boundary Bleeding** a form of inter-channel contamination that can occur during the Discrete Cosine Transform (DCT) quantization phase of the video encoding. By isolating the faces, the system prevents high-frequency data from one view from corrupting the reconstruction of an adjacent one.

2) Chroma Sub-sampling and Planar Composition : The assembly of the Super-Frame represents the final structural transformation, where the discrete spatial data of the G-Buffer is mapped into a single, contiguous memory buffer. The architecture is fundamentally built upon the I420 Planar standard, a specific implementation of the YUV 4:2:0 color space designed for high-efficiency video encoding.

The power of this standard lies in the principle of Chroma Sub-sampling, which allows the system to decouple the Luminance (Y) channel as a high-precision carrier for geometric structure and occupancy—from the Chrominance (U, V)—which stores the photometric (color) attributes. This decoupling is a strategic choice for volumetric streaming:

- The Y Plane (Luminance): Mathematically, this plane maintains a $1:1$ spatial mapping with the original 3D projection, storing the fundamental signal intensity as an 8-bit value $Y \in [0, 255]$. By preserving the full resolution ((WY$\times H$Y), it carries the high-frequency spatial details for the Geometry, Texture, and Occupancy maps.

$$W_Y = (W \cdot 4) \cdot 3 = 12W \tag{6}$$

$$H_Y = H \cdot 3 = 3H \tag{7}$$

- The U and V Planes (Chrominance): Mathematically, these planes are downsampled by a 2×2 factor, reducing the data area to $1/4$ per plane relative to the Luminance. While they carry the C_b and C_r components for the **Texture** atlas, for **Geometry** and **Occupancy** they are populated with a neutral mid-point constant ($\mu = 128$). Due to the 4:2:0 subsampling factor ($f_{sub} = 0.5$), the dimensions of the chrominance planes are halved in both axes:

$$W_{UV} = \frac{12W}{2} = 6W \quad , \quad H_{UV} = \frac{3H}{2} = 1.5H \tag{8}$$

resulting in two WUV $\times H$UV matrices.

The construction of the final frame follows the the I420 Planar layout. In this configuration, the planes are appended vertically in a specific order: first the full-resolution Y-plane, followed by the sub-sampled U and V planes. To maintain a consistent global width (W_{Total}) across the entire buffer—which is a requirement for hardware-accelerated DMA (Direct Memory Access) transfers—the system places the U and V planes side-by-side horizontally.

Super-Frame $= [\text{Atlas}^{Y}_{geo} \parallel \text{Atlas}^{Y}_{tex} \parallel \text{Atlas}^{Y}_{occ}] \rightarrow$ [Subsampled U and V Planes]

Given the horizontal juxtaposition of the chrominance planes ($W_{UV} + W_{UV} = 6W + 6W = 12W$), the global width remains identical to that of the Y-plane. However, the total height (H_{Total}) is the sum of the luminance height and the sub-sampled chrominance height:

$$W_{Total} = 12W$$

$$H_{Total} = H_Y + H_{UV} = 3H + 1.5H = 4.5H$$

Applying the system's target resolution ($W = 640, H = 480$): $W_{Total} = 12 \times 640 = \mathbf{7680}$ px $H_{Total} = 4.5 \times 480 = \mathbf{2160}$ px

The vertical stacking of the planes into a single linear memory block is critical for the GStreamer pipeline to interpret the raw bytes correctly as a valid video frame. Two element must be examinated:

Virtual Frame Height To accommodate the complete I420 planar structure, the system treats the buffer as having a total "virtual" height of 2160 pixels. This value represents the $1.5\times$ height multiplier ($H_Y \times 1.5$) required for I420 packing:

$$H_{Virtual} = H_Y + \left(\frac{H_Y}{2}\right) = 1440 + 720 = 2160 \quad (9)$$

- **Pipeline Compatibility:**
  By defining this virtual dimension, the engine ensures that the FFmpeg subprocess can access the chrominance data at the correct memory offsets through the `stdin` pipe. The entire volumetric dataset—comprising 18 synchronized patches—is thus encapsulated into a standard high-definition panoramic stream, allowing for seamless UDP transmission and real-time synchronization between the Encoder and the Client.

F. Summary

Together, the Geometry, Texture, and Occupancy maps constitute a complete, synchronized 2D descriptor of the volumetric data. By aligning these layers into a unified Super-Frame based on a native I420 (YUV 4:2:0) planar architecture, the encoder effectively translates an unorganized 3D point cloud into a bitstream that is natively compatible with hardware video decoders.

This representation enables the system to leverage high-efficiency compression standards—specifically H.265 (HEVC)—to exploit both temporal and spatial redundancies through a stable, fixed-layout mosaic. By utilizing the Luminance (Y) channel for spatial structure (Geometry and Occupancy) and the Chrominance (U/V) channels for visual attributes (Texture), the approach achieves high compression ratios while preserving the geometric and photometric fidelity of the original scene. Ultimately, this "MPEG-emulation" strategy ensures a fluid, real-time interactive experience by maintaining a consistent 30 FPS pipeline through hardware-accelerated encoding and low-latency UDP streaming.

V. Codec H.265

With the construction of the Super-Frame, the spatial data has been successfully translated into a format compatible with standard digital infrastructure. The following sections detail the final stages of the pipeline, focusing on how this 2D representation is compressed for transmission and how the resulting quality is measured.

A. h.265 structure

The core of the transmission pipeline is the H.265 (High Efficiency Video Coding) engine. This standard was selected for its superior ability to compress complex, high-resolution mosaics—such as the 7680×1440 Atlas Super-Frame—while maintaining high fidelity and a manageable bitrate.

The encoding process is structured into four principal phases, optimized to translate the planar I420 Atlas into an efficient binary bitstream:

- **Phase 1: Prediction**
  The goal is to reduce data redundancy by predicting pixel values before transmission.
  - **Intra-prediction (Spatial):** The encoder predicts blocks based on adjacent pixels within the same frame. This is highly efficient for the Geometry Map, as the projected surfaces of the point cloud typically exhibit smooth and consistent depth gradients.
  - **Inter-prediction (Temporal):** This phase exploits similarities between consecutive frames through Motion Estimation. Thanks to the Fixed-Layout Atlas strategy, a static 3D point remains at the exact same 2D pixel coordinate across frames. This allows the codec to maximize temporal redundancy, transmitting only the residual changes (e.g., small surface movements) rather than re-encoding the entire structure.
- **Phase 2: Transform (DCT/DST)**
  Once the "residual" (the difference between the actual data and the prediction) is obtained, the system applies a Discrete Cosine Transform (DCT). This operation shifts the data from the spatial domain to the frequency domain. In the context of the Atlas, this separates fine textural details (high frequencies) from the broader, uniform geometric areas (low frequencies).
- **Phase 3: Quantization**
  Quantization is the primary lossy stage and a significant factor in the precision of the reconstructed 3D model. Transform coefficients are divided by a Quantization Parameter (QP) and rounded.
  - In the *Geometry Map*, excessive quantization can lead to depth precision errors, manifesting as "jitter" or spatial shifting in the 3D domain.
  - In the *Texture Map*, it results in color degradation and loss of visual fidelity.
  - System Countermeasure: The black gaps in the Cross-Shaped Grid act as buffers to mitigate "boundary bleeding" during this phase, preventing high-

frequency noise from one patch from contaminating adjacent views.

- **Phase 4: Entropy Coding (CABAC)**
  Finally, the quantized data are converted into a lossless binary bitstream using CABAC (Context-Adaptive Binary Arithmetic Coding). This stage eliminates statistical redundancies within the symbols to produce the final, lightweight stream dispatched via UDP/RTP.

a) Hybrid Coding and Stream Configuration: The implementation utilizes a Hybrid Video Coding architecture, which combines spatial and temporal compression to achieve maximum efficiency. To bridge the gap between the internal encoding logic and real-time network transmission, the system utilizes a high-performance FFmpeg-based engine.
FFmpeg is a comprehensive suite of libraries and tools designed for high-speed multimedia processing, acting as the primary computational engine for both compression and streaming.

The system utilizes a direct-feed UDP streaming engine powered by FFmpeg, specifically optimized for low-latency delivery. The effectiveness of the H.265 compression described in previous sections depends on the precise tuning of the FFmpeg command-line parameters that define the encoding behavior and network encapsulation:

- **Encoder (FFmpeg/HEVC):**
  The system utilizes the `libx265` software codec, chosen for its high-fidelity compression of 3D-derived luma data.
- **Low-Latency Tuning:**
  The engine is configured with the `ultrafast` preset and `zerolatency` tuning. This configuration disables internal frame buffering within the encoder, ensuring that the 3D-to-2D projected data is transmitted with the smallest possible motion-to-photon delay.
- **GOP Management:**
  A Group of Pictures (GOP) of 15 frames is enforced (`-g 15`). This ensures that a complete I-frame is transmitted every 0.5 seconds, providing a periodic "spatial reset" that allows the client to recover the 3D structure even in the presence of UDP packet loss.
- **Payload and Protocol:**
  The H.265 bitstream is encapsulated in an MPEG-TS container and streamed via UDP. The bitstream is forced into a grayscale pixel format (`-pix_fmt gray`) to treat the YUV I420 super-frame as a raw intensity map, preserving the mathematical integrity of the depth information.

VI. Real-time Performance Telemetry

Beyond the static analysis of reconstructed matrices, the system implements a comprehensive Real-time Telemetry engine. This monitoring framework is essential for assessing the performance of the pipeline in a live streaming scenario, where latency and synchronization are as critical as visual fidelity.

A. Temporal and Network Metrics

The system continuously tracks a set of Key Performance Indicators (KPIs) through a dedicated asynchronous logging thread. These metrics are stored in high-performance data structures (using the Polars library) for post-session evaluation:

- Latency Analysis: The encoder measures Partial Latency ($t_{recv} - t_{send}$) and End-to-End Latency to identify bottlenecks in the transmission or processing stages.
- Node Processing Times: The system breaks down the time spent in each phase: *Decoding*, *Rendering*, and *Encoding*. This allows for a granular assessment of the computational overhead introduced by the Numba-optimized kernels.
- Throughput and Bitrate: Network performance is monitored by calculating the effective throughput (MB/s) and the output bitrate (Mbps) of the H.265 stream, ensuring the system remains within the bandwidth constraints of the UDP channel.

B. Geometric Integrity Monitoring

In addition to temporal metrics, the encoder performs a live geometric audit of every frame. This is achieved through two specialized diagnostic tools:

- Bounding Box Reporting: The system compares the Axis-Aligned Bounding Box (AABB) of the raw source data with the normalized output. This ensures that the object-centric transformation is correctly applied and that no spatial clipping occurs during real-time rotations.
- Patch Density Analysis: A dedicated thread calculates the number of points projected onto each of the six faces (Patches) of the G-Buffer. This "visibility report" identifies if certain views are under-utilized or if the subject is partially occluded, providing vital feedback on the quality of the 360-degree representation.

C. Diagnostic Logging and Synchronization

To minimize the impact on the main processing loop, all telemetry data is handled via an asynchronous CSV Logger Thread. By offloading the I/O operations, the encoder can maintain a steady frame rate (30 FPS) while recording detailed metadata for every frame. This log acts as a "flight recorder" for the experiment, allowing researchers to correlate specific network events or processing spikes with the visual distortions observed in the reconstructed output.

VII. Decoder

VIII. Introduction(Sort of Abstract)

The Decoder represents the final node of the streaming pipeline. While the Encoder serves as a "virtual director" translating volumetric scenes into a structured video format, the Decoder functions as a high-performance Volumetric Reconstruction Engine. Its primary objective is to interpret the compressed 2D video stream and architecturally rebuild the original 3D environment in real-time. Moving beyond a passive viewer, the Decoder is designed as an active interactive terminal with a dual mission:

- Volumetric Restoration (The Rebuild) The system ingests the incoming H.265/HEVC video stream and initiates a complex inverse projection process.
- Bi-Directional Feedback and Control The Decoder provides the primary interface for user interaction, transforming the stream into a responsive environment. It captures and orchestrates real-time control commands through two distinct feedback channels:
  - Pose Orchestration: Interactive commands such as Yaw, Pitch, and Zoom are captured and transmitted back to the upstream Encoder nodes via dedicated TCP sockets. This allows the user to dynamically influence the "Virtual Stage" and the rendering perspective.
  - Temporal Synchronization: The system manages a Temporal Skip function bound to the pipeline's flow-control node (Skip node), allowing the user to navigate the temporal timeline of the volumetric sequence.

  By leveraging established video standards and high-performance computing libraries, the Decoder effectively bridges the gap between traditional 2D streaming and immersive, interactive 3D visualization.

IX. Stream Ingestion and Video Decoding

The entry point of the decoding pipeline represents a critical architectural transition where the system shifts from the reliable TCP connections used by upstream nodes to a high-speed, low-latency UDP bridge. To ensure maximum data flow continuity and prioritize real-time delivery, the system utilizes port 6000 to receive the incoming signal.

A. Network Reception and RTP/UDP Protocol Handling

While the previous nodes in the pipeline may utilize TCP to ensure data integrity during initial processing, the bridge between the Encoder and the Client relies on the User Datagram Protocol (UDP). This choice is fundamental for real-time interactive scenarios, as it bypasses the inherent delays of retransmission mechanisms that would otherwise stall the stream during network congestion. As the signal arrives via UDP, it is processed directly by the FFmpeg engine , which acts as a transparent translator. FFmpeg automatically handles the network headers and reassembles the original H.265 bitstream from the incoming packets. To ensure stability, the system uses the large FIFO and network buffers defined in the connection string to reorder any misaligned data and prepare a clean, structured bitstream for the 3D reconstruction sequence."

X. FFmpeg Stream Ingestion and Decoding

The Decoder utilizes the FFmpeg engine as the fundamental bridge between the network and the volumetric reconstruction environment. By leveraging the OpenCV `CAP_FFMPEG` backend, the system provides a highly integrated stream ingestion and decoding flow optimized for high-resolution volumetric data. The reception and decoding process follows these optimized stages:

- **High-Capacity Network Ingestion:**
  The system opens a UDP socket using a specialized URL to configure FFmpeg's internal buffer management. To prevent packet loss during the transmission of the high-resolution Atlas Super-Frame, the system implements massive buffers: a FIFO size of 50 MB (`fifo_size=50000000`) and a network buffer of 100 MB (`buffer_size=100000000`). These settings absorb network jitter and fluctuations, ensuring a steady data flow to the decoder.
- **Raw Data Integrity and YUV Preservation:**
  A critical feature of this implementation is the **avoidance of RGB/BGR color space conversion**. Standard video players convert YUV bitstreams to RGB for human visualization; however, in this system, the pixels represent mathematical coordinates rather than aesthetic colors.
  Since the Luminance (Y) channel carries the **Geometry Map** (depth) and the **Occupancy Map**, any color conversion would introduce floating-point rounding errors. Even a ± 1 change in pixel intensity would translate to a spatial shift in the 3D domain.
  By maintaining the raw **YUV/Planar format**, the system ensures that the depth values remain bit-accurate.
- **Direct Frame Handover (`cap.read()`):**
  The decoded Super-Frame is delivered to the Python environment as a raw NumPy matrix. This direct handover bypasses unnecessary post-processing, providing the high-precision intensity values directly to the Numba-optimized kernels for immediate spatial back-projection.
- **Direct Frame Handover (`cap.read`):** The decoded Super-Frame is delivered to the Python environment as a raw NumPy matrix. This direct handover bypasses unnecessary post-processing, providing the high-precision intensity values directly to the **Numba-optimized kernels** for immediate spatial back-projection.

$$\text{UDP Packets} \xrightarrow{\text{FFmpeg Buffers}} \text{FFmpeg (Raw YUV)} \xrightarrow{\text{cap.read()}} \text{Raw Matrix} \xrightarrow{\text{Nu}} \tag{10}$$

A. Bitstream Decoding and Super-Frame Recovery

Once the stream is established, the FFmpeg backend processes the H.265 signal at a steady 30 FPS. The final output of this phase is the restoration of the original Super-Frame, a massive mosaic measuring 7680×1440 pixels. This frame serves as a synchronized spatial anchor, housing the Geometry, Texture, and Occupancy data required to rebuild the volumetric point cloud.

XI. Super-Frame Decomposition

Once the Super-Frame is recovered, the Decoder must transition from treating the data as a single video image to treating as three distinct, synchronized logical layers.

A. Spatial Unpacking and Atlas Extraction

The system initiates the deconstruction by performing the precise inverse of the aggregation process used during the encoding stage. The Super-Frame is not treated as a single

image, but as a contiguous memory block that is sliced into three equal segments. This partitioning effectively separates the consolidated signal into three synchronized logical layers: the Geometry, Texture, and Occupancy Atlases. The unpacking process operates as follows:

- Geometry and Occupancy Extraction: These layers are retrieved directly from the first and third segments of the Y-plane. Since their chrominance components were nulled during encoding, the decoder ignores the corresponding U and V offsets, significantly reducing memory bandwidth during the reconstruction of the spatial structure.
- Texture Extraction: This layer requires a full planar reconstruction. The system extracts the central pixel segment from the Y-plane and combines it with the underlying subsampled U and V planes to restore the photometric attributes of the scene.

Despite this logical separation, each Atlas remains organized in the deterministic 3×4 cross-shaped grid established at the source. This structural consistency ensures that the spatial relationships between the six cardinal views are perfectly preserved. Consequently, the subsequent reconstruction kernels can use static memory offsets to simultaneously access depth, color, and occupancy for any given 3D projection.

B. Artifact Mitigation and Morphological Erosion

A critical challenge in this phase is the presence of "bleeding" or "ringing" artifacts introduced by the H.265 quantization process. These compression artifacts often manifest as noisy pixels at the sharp boundaries of the object's silhouette. To prevent these errors from being reconstructed as floating 3D "ghost" points, the Decoder applies a morphological erosion filter to the Occupancy Map. By using a 2×2 kernel, the system shrinks the object's detected outline by 1–2 pixels, effectively "stripping away" the corrupted perimeter data and ensuring that only the high-fidelity core of the volumetric object is passed to the reconstruction stage.

XII. Volumetric Reconstruction: 2D to 3D

This section describes the mathematical core of the Decoder: the process of translating decoded pixel data back into a volumetric environment. This stage reverses the projection logic established by the Encoder to architecturally rebuild the point cloud from its 2D spatial descriptors.

A. Inverse Orthogonal Projection Framework

The reconstruction process begins by iterating through the six cardinal patches stored within the Y-plane of the Geometry and Occupancy Atlases. For every active pixel (u, v) identified by the Occupancy Map, the system extracts its normalized depth value d from the corresponding sector in the Geometry Atlas.

It is crucial to note that depth information is stored exclusively in the Luminance (Y) channel, ensuring that the spatial structure is preserved at full resolution ($W \times H$) without the degradation of chroma subsampling. The 8-bit depth value $Y_{geo}(m, n)$ is converted into a normalized spatial coordinate:

$$d_{norm} = \frac{Y_{geo}(m, n)}{255.0}$$

The system then applies a conditional mapping logic based on the cardinal orientation of the patch. Given the normalized pixel coordinates u_{norm} and v_{norm}, the 3D position $\mathbf{P} = [nx, ny, nz]^T$ is derived. For example, for the Front View (ID 0):

$$nx = u_{norm}, \quad ny = 1.0 - v_{norm}, \quad nz = d_{norm}$$

B. Attribute Restoration and Texture Mapping

Simultaneously with the spatial reconstruction, the system performs *Attribute Restoration* to recover the visual appearance of each point. Unlike traditional RGB streams, the input buffer follows the I420 Planar standard, requiring a real-time color conversion logic within the reconstruction kernel.

XIII. Interactive Control and Feedback Loop

Moving beyond the role of a passive receiver, the Decoder functions as an active control terminal, facilitating a high degree of bi-directional interactivity within the volumetric stream. By intercepting user inputs and relaying them to upstream nodes, the system effectively closes the operational loop between the "Virtual Architect" and the "Virtual Director," ensuring that the reconstructed environment responds dynamically to user intent. The pixel position (u, v) used for geometry serves as a common spatial anchor for the Texture Atlas. However, retrieving the color requires sampling from three different memory offsets:

- **Luminance (Y):** Extracted at full resolution from the Texture sector of the Y-plane.
- **Chrominance (U, V):** Since these planes are subsampled, the kernel retrieves the U and V values at coordinates $(u/2, v/2)$ from their respective subsampled matrices.

To comply with the standard RGB requirements of the PLY format and visualizers (such as Open3D), the kernel applies the inverse BT.601 conversion matrix:

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1.164 & 0.000 & 1.596 \\ 1.164 & -0.392 & -0.813 \\ 1.164 & 2.017 & 0.000 \end{bmatrix} \begin{bmatrix} Y - 16 \\ U - 128 \\ V - 128 \end{bmatrix}$$

The final output is a spatially accurate point cloud where every vertex carries its original photometric data.

A. Command Orchestration and Mapping

The Decoder translates raw user interactions into semantic control signals through a non-blocking keyboard listener. This interface allows for real-time manipulation of the volumetric scene, mapping specific keys to spatial and temporal parameters:

- Spatial Navigation (Pose): The keys w/s (Pitch) and a/d (Yaw) allow the user to rotate the object-centric reference frame. The +/- keys manage the Zoom factor, effectively adjusting the "Virtual Camera" distance within the projection volume. These navigation commands are specifically

routed to Encoder B, which serves as the primary execution node for spatial adjustments. Upon receiving these signals, the encoder dynamically re-calculates its projection matrices, ensuring that the subsequently generated 2D Atlases align with the user's updated perspective.
- Temporal Navigation (Skip): The keys j/l are dedicated to the Temporal Skip function, allowing the user to step forward or backward through the point cloud sequence. Unlike the spatial commands sent to the encoder, these inputs are bound to the SKIP node, which acts as the flow-control authority for the binary stream, enabling non-linear navigation through the point cloud sequence.

B. Multi-Channel Feedback Architecture

To ensure that these commands are executed with minimal latency, the Decoder manages a bi-lateral communication framework utilizing persistent TCP sockets. Each channel is bound to a specific node in the pipeline to handle different aspects of the interactive session:

- Control_B (Spatial Feedback): Pose and Zoom commands are transmitted directly to Encoder B. This allows the encoder to adjust the projection matrices in real-time, ensuring that the next set of generated 2D Atlases reflects the user's chosen perspective.
- Control_SKIP (Temporal Feedback): Skip commands are routed to the SKIP node, which acts as the flow-control authority for the binary stream. This node dictates which frame ID is ingested by the pipeline, enabling non-linear navigation of the dataset.

C. The Interactive Feedback Loop

The synergy between these channels creates a low-latency Interactive Feedback Loop. When a key is pressed, the Decoder records a high-precision timestamp and dispatches the command. As the upstream Encoder updates its rendering state and transmits the new Super-Frame, the Decoder identifies the arrival of the updated data. This bi-directional flow allows the system to prioritize user intent, where the client's choices at the end of the pipeline directly dictate the computational behavior at the start, bridging the gap between remote streaming and local interactivity.

XIV. Real-time QoE Telemetry and Data Fusion

The Decoder serves as the primary diagnostic hub of the entire pipeline, as it is the terminal node where the ultimate Quality of Experience (QoE) is realized and recorded. Being the final stage of the transmission chain, it is uniquely positioned to evaluate the cumulative impact of network jitter, processing overhead, and synchronization errors on the final 3D reconstruction.

A. Performance Metrics and Live Monitoring

To maintain a high-fidelity interactive stream, the system implements a high-precision telemetry engine that tracks Key Performance Indicators for every frame received. The telemetry system monitors several critical factors:

- Latency and Jitter Analysis: By measuring the Delta Arrival Time between successive packets, the system calculates instantaneous jitter and identifies potential "freeze" events caused by network congestion.
- Computational Benchmarking: The Decoder breaks down its internal processing cycle into three distinct phases—Read, Decode, and Render—allowing the "Virtual Architect" to pinpoint bottlenecks within the Numba kernels or the hardware decoder.
- Network Throughput: TThe system estimates the effective bitrate and throughput in real-time by cross-referencing the size of the incoming raw YUV/Intensity frames with their specific arrival intervals, ensuring the stability of the H.265 stream.

B. Command Reactivity and Interaction Latency

A specialized diagnostic tool within the telemetry engine measures Reactivity, which is defined as the temporal gap between the dispatch of a user command (such as a Yaw rotation) and the arrival of the first frame that visually reflects that adjustment. This "Command-to-Photon" latency is the most critical metric for assessing interactive fluidity. It accounts for the total round-trip time, encompassing the TCP control signal's journey, the Encoder's re-projection and rendering time, and the subsequent UDP video transmission back to the client.

C. The "Super-Merge" and Frame Fate Analysis

The telemetry process concludes with an offline Data Fusion operation known as the Super-Merge. Upon session completion, the Decoder unifies its local telemetry logs with the data recorded by all other nodes in the pipeline, including the Camera, Skip, and both Encoder nodes. This analysis utilizes an "As-Of Join" strategy to synchronize disparate timestamps across various hardware clocks.

The result of this fusion is a comprehensive Frame Fate report. This report tracks the entire lifecycle of a frame—from its initial capture at the camera to its final reconstruction at the client—allowing researchers to identify exactly where and why a frame might have been lost or delayed, whether due to a voluntary "Skip" command, a TCP buffer overflow, or a UDP transmission error. This provides a complete narrative of the volumetric stream's journey, closing the gap between raw data transmission and meaningful performance analysis.

XV. Optional:Web-Based Real-Time Monitoring

Beyond the local reconstruction, the system includes an optional Web Telemetry Bridge designed for remote observation and live diagnostics. This component is essential for scenarios where the "Virtual Architect" needs to be monitored by external observers or for recording performance trends across long-duration sessions.

A. Interactive Dashboard and Visual KPIs

The front-end of this system consists of a web-based dashboard that visualizes the "health" of the volumetric pipeline in real-time. This interface provides several key functions:

- Live KPI Streams: Observers can view dynamic charts representing the Command Reactivity and the Frame Arrival Delta, allowing for immediate identification of ”stuttering” or high-latency events.
- Session Status Tracking: The dashboard displays the current state of the feedback loop, showing active Yaw, Pitch, and Zoom values as they are transmitted to the Encoder nodes.
- Remote Event Logging: Any significant errors, such as a ”Socket Timeout” or a ”Decoder Reset,” are instantly pushed to the web console, providing a remote audit trail that complements the local ”Super-Merge” analysis.

By decoupling the visualization of metrics from the 3D rendering engine, the system achieves a robust monitoring environment that scales with the complexity of the volumetric data without compromising the low-latency requirements of the end-user experience.

XVI. Numerical results

copia e incolla da appunti di Maria Giovanna
article [utf8]inputenc [english]babel graphicx amsmath booktabs array placeins float

XVII. Coherence of the Geometric Context

A reliable error analysis presupposes a coherent geometric pipeline: each transformation, from the native voxel grid to the client-side reconstruction, must preserve the geometry of the cloud. Evaluating the process stage by stage, we verify that every spatial transformation behaves as expected and that the cloud received by the client is geometrically equivalent to the one transmitted.

All numerical values reported here are taken from a single control run executed at a target bitrate of 10 Mbps, with a packet size of 1000 bytes and a rendering resolution of 1024×768. The run conditions and the per-frame geometric measurements are logged independently by the encoder and by the client, so the expected (encoder-side) and measured (client-side) quantities are synchronised frame by frame.

To illustrate the system's behaviour, two reference frames are used:

- **Frame 260**: a *representative* frame with *regular* behaviour.
- **Frame 255**: a frame with *anomalous* behaviour.

A. Pipeline Architecture and Native Voxel Space

The input dataset (*loot*, `vox10`/8i sequence) represents the subject as a set of points with integer coordinates quantised on 10 bits; consequently, each spatial axis (X, Y, Z) spans the range $[0, 1023]$. This constitutes the native voxel cube, i.e. the maximum theoretical extent of the digitised scene. Inside this cube the human figure occupies only a portion: the subject spans $X \in [28, 381]$, $Y \in [5, 1013]$, $Z \in [124, 471]$, i.e. about $353 \times 1008 \times 347$ voxels, an upright figure developed predominantly along the vertical axis Y. A world-scale factor of 0.181985 stored in the header converts voxels to world units; applied to the ~ 1008-voxel height, it yields ≈ 1.83 world units, consistent with a real human stature.

The geometric data passes through three transformations—pose normalisation, cube projection and, on the receiver, client-side reconstruction—with transmission and coding taking place between projection and reconstruction.

a) Pose Normalisation.:
The transformation proceeds in a few steps: each raw voxel v is centred on the barycentre b, rescaled by `final_scale`, rotated by the pose R (yaw and pitch), and then translated along Z so that the subject lies in front of the camera:

$$p = \texttt{final\_scale} \cdot R \cdot (v - b) + [\,0,\, 0,\, 1200\,], \qquad (11)$$

where p is the resulting normalised point, `final_scale` is the pose-normalisation scale factor and R is the rotation defined by the pose (yaw, pitch). The bounding box is logged both before (pre-norm) and after (post-norm) this step. On the control run the pose is neutral (yaw $=$ pitch $= 0$), so R is the identity and the two boxes coincide, confirming that normalisation introduces no spurious deformation.

b) Cube Projection.:
The normalised cloud is projected onto the faces of a unit cube: each point p is encoded into a stored depth value `geo`, a pixel index m and a normalised cube coordinate n,

$$\texttt{geo} = \mathrm{round}(d_{\text{norm}} \cdot 255)\,, \qquad (12)$$

$$m = \mathrm{round}(u_{\text{norm}} \cdot (M-1))\,, \qquad (13)$$

$$n = \frac{p - c_{\text{bb}}}{\texttt{diag} \cdot s_{\text{global}}} + 0.5, \qquad (14)$$

where $d_{\text{norm}}, u_{\text{norm}} \in [0, 1]$ are the normalised depth and in-face position of the point, M is the number of pixels along a face axis, `diag` and c_{bb} are the size and centre of the bounding box, and s_{global} is the cube scale.

c) Client-Side Reconstruction.:
The client applies the inverse of each mapping to rebuild the cloud:

$$d_{\text{norm}} = \frac{\texttt{geo}}{255}, \qquad (15)$$

$$u_{\text{norm}} = \frac{m}{M-1}, \qquad (16)$$

$$\hat{p} = (n - 0.5) \cdot \texttt{diag} \cdot s_{\text{global}} + c_{\text{bb}}. \qquad (17)$$

Concretely, the client reads back the three transmitted maps and reverses every encoding step: the stored value is rescaled to a depth in $[0, 1]$, the pixel index to a normalised face position, and the cube coordinate to a metric 3D point $\hat{p}$. Repeating this for every pixel regenerates the full point cloud, which reproduces the transmitted geometry up to the quantisation introduced by the 8-bit depth and the integer pixel grid.

B. Stage-by-Stage Verification

The comparison between expected and measured values for the analysed frame is summarised in Table I; spatial quantities are expressed in the normalised world reference frame, depth values in native voxel units.

Quantity	Objective	Expected		Measured
Global scale	Scale consistency	s_{global}	=	$s_{\text{global}} = 0.6425$
		0.6425		
Final scale	Pose scaling	0.4545		0.4545
Pose (yaw, pitch)	Neutral pose	$(0, 0)$		$(0, 0)$
BB centre	Centre preservation	$(-3.80, -4.15, 1229.62)$		$(-3.80, -8.02, 1229.62)$
BB extent	Size preservation	$161\times449\times188$		$171.7\times462.5\times190.3$
Radius of gyration	Branch independence	281.07		281.07
Front-face depth min	Depth-map sanity	—		0.380
Depth quantisation	Depth fidelity	Floor		Mean error ≈ 2.0 vox
		≈ 2.6 vox/level		

TABLE I
COMPARISON OF EXPECTED AND MEASURED VALUES FOR THE PIPELINE VERIFICATION.

The comparison confirms that the pipeline preserves the geometry end to end. The cube scale and the radius of gyration are identical on the two sides, so the reconstruction branches operate symmetrically and add no distortion; the centre transmitted by the encoder is also recovered at the client to within $\Delta < 0.05$, which locates the residual discrepancy in the reconstruction rather than in the transport. Depth stays within its theoretical limit: the mean spatial deviation (≈ 2.0 vox) remains below the 8-bit quantisation floor (≈ 2.6 vox/level), and a regular front-face depth minimum (0.380) rules out spurious deep points. The only differences are a small offset of the reconstructed centre, and a slight extent inflation, Δ, both expected discretisation artefacts of the cube projection and voxel repositioning. The transmitted and reconstructed clouds are therefore geometrically equivalent.

C. Anomalous Frame Analysis

Validation over the sequence revealed a limited number of anomalous frames. The key observation is that the anomaly is *not* present on the encoder side: for the analysed frame the encoder bounding box is regular and the transmitted centre is in line with the neighbouring frames. The anomaly appears only in the reconstruction, where the Z centre collapses well below its typical value and the front-face depth minimum drops sharply, indicating a spurious point pushed far in depth.

Crucially, the radius of gyration is again identical between the two reconstruction branches: both branches rebuild the same anomalous geometry, so the decoded stream itself faithfully carries the outlier already present in the source frame. The anomaly is therefore single-frame reconstruction noise, not a transmission or decoding defect. Table II contrasts the regular and anomalous frames on the client side.

Indicator	Regular frame	Anomalous Frame
Centre offset $\lvert\Delta Z\rvert$	1.59	57.78
Reconstructed Z centre	1229.62	1170.19
BB extent (X, Y, Z)	$(171.7, 462.5, 190.3)$	$(282.5, 475.7, 328.7)$
Front-face depth min	0.380	0.161

TABLE II
CONTRAST BETWEEN REGULAR AND ANOMALOUS FRAMES ON THE CLIENT SIDE.

Outcome
Identical
Identical
Pre/post coincide
$\Delta \approx (10.8, 13.9, 2.6)$
$\Delta < 10^{-4}$
Regular
Below floor

XVIII. ERROR ANALYSIS

A reliable system is expected not only to be geometrically coherent but also to degrade predictably: the reconstruction error should be consistent and follow the trends typical of rate–distortion behaviour. Whereas the previous analysis confirmed that the geometry is preserved in form, the error analysis measures how faithfully it is preserved in magnitude. As the encoding rate decreases, a loss of fidelity is unavoidable; the relevant question is how gracefully the reconstruction degrades. To capture this from complementary angles, the error is examined at three levels: first on the coded signal itself, and then on the reconstructed geometry. At the signal level, fidelity is assessed directly on the depth map, before any 3D reconstruction takes place (Section XVIII-A). At the geometric level, fidelity is assessed on the reconstructed point cloud, from two viewpoints: in 3D, through the distance to the reference geometry (Section XVIII-B), and in 2D, through the distance between the projections of corresponding points (Section XVIII-D).

A. Map Coding Error (Depth PSNR)

At the first level the error is measured directly on the depth map, before 3D reconstruction, as the error between the original and the encoded map at each rate. The comparison is restricted to the luma channel, since the chroma planes carry no depth information and would otherwise bias the result. Three complementary indicators are reported versus rate: MSE, PSNR, and SSIM, all computed on the luma channel and averaged over the 300 frames of the sequence at each of the tested rates.

Across this range the indicators degrade consistently and without anomalies: MSE decreases monotonically as the rate increases, while PSNR and SSIM increase monotonically, in agreement with the expected rate–distortion behaviour.

The improvement is most pronounced at the lowest rates and attenuates as the rate increases: PSNR gains about 12.8 dB between 1 and 5 Mbps, but only a further 6.2 dB between 5 and 10 Mbps, while MSE drops by a factor of 15 and then by a further factor of about 4 over the same two intervals. SSIM follows the same pattern, rising from 0.655 at 1 Mbps to above 0.99 already at 5 Mbps, after which it approaches its upper bound and the margin for further improvement narrows accordingly. This pattern is consistent with a system that allocates the available bits efficiently at low rates and reaches diminishing returns once most of the depth-map fidelity has already been recovered.

B. 3D Error

At the second level the error is measured in 3D, after reconstruction, as the distance between the reconstructed and the reference point clouds. The analysis is carried out at two granularities: a *global* one, over the whole cloud, where the overall distortion is summarised by four complementary metrics; and a *point-wise* one, where the error is localised on a set of anatomical landmarks. Both rely on the same initial alignment step, evaluated under two strategies—a classical and

a robust method. The alignment procedure, the global metrics it enables, and the two strategies are described first; the point-wise analysis follows.

1) Iterative Closest Point Algorithm: Before any distance can be measured between the reconstructed point cloud and the reference, the two must be expressed in the same frame of reference. Reconstruction and decoding can introduce a small rigid offset between the two point clouds—a translation, a rotation, or both—which is not, in itself, a geometric error: it merely reflects a difference in pose, not in shape. Measuring distances without correcting for this offset would conflate alignment error with reconstruction error.

This correction is performed via the Iterative Closest Point (ICP) algorithm. Given a reference point cloud P and a reconstructed point cloud Q, ICP estimates the rigid transformation—a rotation R and a translation $\mathbf{t}$—that minimises the sum of squared distances between each point of Q and its closest point in P:

$$(R^*, \mathbf{t}^*) = \arg\min_{R,\,\mathbf{t}} \sum_i \left\| R\, q_i + \mathbf{t} - p_{\sigma(i)} \right\|^2, \qquad (18)$$

where $p_{\sigma(i)}$ denotes the closest point in P to the transformed point q_i. The algorithm alternates between finding these closest-point correspondences and updating $(R, \mathbf{t})$, until convergence.

In practice, ICP restricts candidate correspondences to a maximum search radius around each point, since allowing arbitrarily distant matches would let the algorithm anchor to unrelated structures in the scene. For the sequences analysed here—a human subject approximately 1.8 m tall—a radius of 200 mm was used: wide enough to recover the correct correspondence even if the reconstructed cloud is offset by 10–15 cm, yet narrow enough to exclude floating artefacts elsewhere in the scene.

Once $(R^*, \mathbf{t}^*)$ is found, it is applied to the entire reconstructed cloud, and all subsequent error metrics are computed on the aligned result.

2) Error Metrics: Once the point clouds are aligned, the geometric distortion is quantified using four complementary metrics: Mean Error, Root Mean Square Error (RMSE), Chamfer distance, and Hausdorff distance(worst case). These metrics are built upon the fundamental point-to-cloud distance. Given a point $q \in Q$ and a reference cloud P, the distance is defined as:

$$d(q, P) = \min_{p \in P} \|q - p\|_2. \qquad (19)$$

The **Mean Error** represents the average deviation of the reconstructed points from the reference surface:

$$e_{\text{mean}}(Q, P) = \frac{1}{|Q|} \sum_{q \in Q} d(q, P). \qquad (20)$$

The **RMSE** applies a quadratic penalty, making it more sensitive to larger local errors than the simple mean. As detailed in Section XVIII-B3, its exact formulation differs slightly between the classical and the robust alignment method; in its general, bidirectional form it is given by:

$$e_{\text{RMSE}}(Q, P) = \sqrt{\frac{1}{|Q| + |P|} \left(\sum_{q \in Q} d(q, P)^2 + \sum_{p \in P} d(p, Q)^2 \right)}. \qquad (21)$$

The **Chamfer distance** provides a symmetric evaluation by averaging the bidirectional distances, capturing both the points in Q that diverge from P and the points in P that are not accurately reconstructed in Q:

$$d_{\text{Chamfer}}(Q, P) = \frac{1}{|Q|} \sum_{q \in Q} d(q, P) + \frac{1}{|P|} \sum_{p \in P} d(p, Q). \qquad (22)$$

Finally, the **Hausdorff distance** measures the worst-case scenario, identifying the maximum local error across the entire geometry:

$$d_{\text{Hausdorff}}(Q, P) = \max \left(\max_{q \in Q} d(q, P), \max_{p \in P} d(p, Q) \right). \qquad (23)$$

3) Classic vs. Robust Alignment: The presence of noise or outliers in the reconstructed point cloud can significantly affect the ICP algorithm. Since ICP minimises a sum of squared distances, a small number of outlying points—arising, for example, from reconstruction artefacts or floating noise in the scene—can disproportionately influence the estimated transformation, pulling the alignment away from the correct pose. Two alignment strategies are compared to assess this effect.

In the **classical method** (coupled mode), ICP is applied directly between the raw reconstructed cloud Q and the reference P: the same point sets are used both to estimate the rigid transformation and to evaluate the resulting error. If Q contains significant outliers, the transformation itself may be biased towards them.

In the **robust method** (decoupled mode), the alignment and measurement phases are decoupled. A statistical outlier-removal filter is first applied to Q, producing a clean subset Q_{clean}; ICP is then performed between Q_{clean} and P, yielding a transformation $(R^*, \mathbf{t}^*)$ that is not influenced by the removed points. This transformation is then applied to the *entire* original cloud Q, including the points discarded during filtering, and all error metrics are computed on this fully aligned, unfiltered cloud. The alignment is therefore stabilised against outliers, while the reported error still reflects every point actually produced by the reconstruction.

One consequence of this decoupling concerns RMSE specifically. In the classical method, RMSE is taken directly from the ICP procedure itself (the *inlier* RMSE returned by the registration step), and is therefore computed only over the point correspondences that ICP retained within its search radius. In the robust method, RMSE is instead computed directly from the bidirectional point-to-cloud distances over the entire aligned cloud, with no points excluded. The two values are consequently not computed over the same point set, and a direct numerical comparison between the classical

and robust RMSE should be interpreted with this difference in mind, rather than as a like-for-like measurement of the same quantity under two alignment strategies.

4) Data Analysis: The two alignment strategies are now compared on the reconstructed sequences, across the tested rates and across three stages of the reconstruction pipeline: the raw encoder output, the pre-erosion reconstruction, and the post-erosion reconstruction. For each combination, the four metrics introduced in Section XVIII-B2 are reported in both their native unit (voxels, the unit in which ICP and all distances are computed) and in the corresponding physical unit (mm/cm), to support both reproducibility and intuitive interpretation of the results.

Table III reports the results obtained with the **classical method**, in which alignment and measurement are performed on the same, unfiltered point cloud.

VOXEL

Metrica	10M	5M	1M
Bitrate Teorico (Mbps)	1.000767e+01	5.852991e+00	1.365472e+00
Bitrate Effettivo (Mbps)	1.110689e+01	6.517669e+00	1.569996e+00
PSNR Teorico Encoder (dB)	5.593692e+01	5.127582e+01	4.016612e+01
PSNR Effettivo Client (dB)	5.593692e+01	5.127582e+01	4.016612e+01
SSIM Teorico Encoder	9.983876e-01	9.960378e-01	8.305009e-01
SSIM Effettivo Client	9.983876e-01	9.960378e-01	8.305009e-01
Errore Encoder (voxel)	8.714898e-14	8.817153e-14	8.760108e-14
Errore Pre-Erosione (voxel)	1.769874e+00	2.022613e+00	3.342245e+00
Errore Post-Erosione (voxel)	1.794366e+00	2.044555e+00	3.366359e+00
RMSE Encoder (voxel)	1.021854e-13	1.032299e-13	1.024562e-13
RMSE Pre-Erosione (voxel)	4.018014e+00	7.424537e+00	2.087434e+01
RMSE Post-Erosione (voxel)	3.680752e+00	6.217585e+00	1.767352e+01
Chamfer Encoder (voxel)	1.742988e-13	1.763431e-13	1.752022e-13
Chamfer Pre-Erosione (voxel)	4.443455e+00	6.027135e+00	1.652100e+01
Chamfer Post-Erosione (voxel)	4.385008e+00	5.755661e+00	1.423871e+01
Hausdorff Encoder (voxel)	2.083920e-13	2.084223e-13	2.065131e-13
Hausdorff Pre-Erosione (voxel)	1.025367e+02	2.172657e+02	6.550690e+02
Hausdorff Post-Erosione (voxel)	8.641861e+01	1.747816e+02	6.214556e+02

MILLIMETRI (fattore 1 voxel = 1.820 mm)

Metrica	10M	5M	1M
Bitrate Teorico (Mbps)	1.000767e+01	5.852991e+00	1.365472e+00
Bitrate Effettivo (Mbps)	1.110689e+01	6.517669e+00	1.569996e+00
PSNR Teorico Encoder (dB)	5.593692e+01	5.127582e+01	4.016612e+01
PSNR Effettivo Client (dB)	5.593692e+01	5.127582e+01	4.016612e+01
SSIM Teorico Encoder	9.983876e-01	9.960378e-01	8.305009e-01
SSIM Effettivo Client	9.983876e-01	9.960378e-01	8.305009e-01
Errore Encoder (mm)	1.586111e-13	1.604722e-13	1.594340e-13
Errore Pre-Erosione (mm)	3.221171e+00	3.681155e+00	6.082085e+00
Errore Post-Erosione (mm)	3.258467e+00	3.721091e+00	6.126774e+00
RMSE Encoder (mm)	1.859774e-13	1.878785e-13	1.864704e-13
RMSE Pre-Erosione (mm)	7.312706e+00	1.351266e+01	3.799130e+01
RMSE Post-Erosione (mm)	6.698988e+00	1.131400e+01	3.216981e+01
Chamfer Encoder (mm)	3.172223e-13	3.209444e-13	3.188679e-13
Chamfer Pre-Erosione (mm)	8.087088e+00	1.096939e+01	3.006823e+01
Chamfer Post-Erosione (mm)	7.980714e+00	1.047530e+01	2.591446e+01
Hausdorff Encoder (mm)	3.792775e-13	3.793208e-13	3.758539e-13
Hausdorff Pre-Erosione (mm)	1.866168e+02	3.954236e+02	1.192225e+03
Hausdorff Post-Erosione (mm)	1.572746e+02	3.181025e+02	1.131049e+03

TABLE III
RESULTS FOR THE CLASSICAL METHOD (COUPLED). LEFT: METRICS EVALUATED IN NATIVE VOXEL UNITS. RIGHT: METRICS SCALED TO MILLIMETRES (1 VOXEL = 1.820 MM) AT 10M, 5M, AND 1M BITRATES.

Table IV reports the corresponding results obtained with the **robust method**, in which the alignment is estimated on an outlier-filtered copy of the cloud and then applied to the complete, unfiltered reconstruction.

VOXEL

Metrica	10M	5M	1M
Bitrate Teorico (Mbps)	1.000767e+01	5.852991e+00	1.365472e+00
Bitrate Effettivo (Mbps)	1.110689e+01	6.517669e+00	1.569996e+00
PSNR Teorico Encoder (dB)	5.593692e+01	5.127582e+01	4.016612e+01
PSNR Effettivo Client (dB)	5.593692e+01	5.127582e+01	4.016612e+01
SSIM Teorico Encoder	9.983876e-01	9.960378e-01	8.305009e-01
SSIM Effettivo Client	9.983876e-01	9.960378e-01	8.305009e-01
Errore Encoder (voxel)	8.977658e-14	8.863080e-14	8.836228e-14
Errore Pre-Erosione (voxel)	1.768993e+00	2.022436e+00	3.347080e+00
Errore Post-Erosione (voxel)	1.789168e+00	2.042580e+00	3.377520e+00
RMSE Encoder (voxel)	1.063243e-13	1.043649e-13	1.043138e-13
RMSE Pre-Erosione (voxel)	2.888337e+00	4.824485e+00	2.009790e+01
RMSE Post-Erosione (voxel)	2.721350e+00	4.086110e+00	1.479031e+01
Chamfer Encoder (voxel)	1.795532e-13	1.772602e-13	1.767246e-13
Chamfer Pre-Erosione (voxel)	4.442278e+00	6.025890e+00	1.651510e+01
Chamfer Post-Erosione (voxel)	4.383452e+00	5.752538e+00	1.423923e+01
Hausdorff Encoder (voxel)	2.108047e-13	2.101085e-13	2.074103e-13
Hausdorff Pre-Erosione (voxel)	1.025529e+02	2.173008e+02	6.550395e+02
Hausdorff Post-Erosione (voxel)	8.643804e+01	1.748286e+02	6.213867e+02

MILLIMETRI (1 voxel = 1.820 mm)

Metrica	10M	5M	1M
Bitrate Teorico (Mbps)	1.000767e+01	5.852991e+00	1.365472e+00
Bitrate Effettivo (Mbps)	1.110689e+01	6.517669e+00	1.569996e+00
PSNR Teorico Encoder (dB)	5.593692e+01	5.127582e+01	4.016612e+01
PSNR Effettivo Client (dB)	5.593692e+01	5.127582e+01	4.016612e+01
SSIM Teorico Encoder	9.983876e-01	9.960378e-01	8.305009e-01
SSIM Effettivo Client	9.983876e-01	9.960378e-01	8.305009e-01
Errore Encoder (mm)	1.633934e-13	1.613868e-13	1.608193e-13
Errore Pre-Erosione (mm)	3.219567e+00	3.680834e+00	6.093141e+00
Errore Post-Erosione (mm)	3.256285e+00	3.717496e+00	6.137986e+00
RMSE Encoder (mm)	1.935102e-13	1.899442e-13	1.898512e-13
RMSE Pre-Erosione (mm)	5.242214e+00	8.780563e+00	3.657818e+01
RMSE Post-Erosione (mm)	4.956513e+00	7.436720e+00	2.611757e+01
Chamfer Encoder (mm)	3.267867e-13	3.226135e-13	3.216387e-13
Chamfer Pre-Erosione (mm)	8.084945e+00	1.096712e+01	3.005749e+01
Chamfer Post-Erosione (mm)	7.977882e+00	1.046962e+01	2.591540e+01
Hausdorff Encoder (mm)	3.895977e-13	3.823976e-13	3.774867e-13
Hausdorff Pre-Erosione (mm)	1.866462e+02	3.954875e+02	1.192172e+03
Hausdorff Post-Erosione (mm)	1.573172e+02	3.181881e+02	1.130924e+03

TABLE IV
RESULTS FOR THE ROBUST METHOD (DECOUPLED). LEFT: METRICS IN NATIVE VOXEL UNITS. RIGHT: METRICS SCALED TO MILLIMETRES (1 VOXEL = 1.820 MM) AT 10M, 5M AND 1M BITRATES.

While the numerical tables provide precise measurements across all configurations, the underlying rate–distortion trends and the distinct behaviours of the two alignment strategies are best observed visually. The four metrics are plotted against the target bitrates to illustrate how the geometric fidelity degrades and how the robust alignment impacts the final error estimation.

As shown in Figure 10, all four metrics confirm the expected rate–distortion trend: geometric distortion increases as the available bitrate is reduced from 10 Mbps to 1 Mbps. Across all plots, the raw encoder output establishes a near-zero lower bound, demonstrating that the majority of the geometric error is introduced during the client-side reconstruction and projection phases rather than by the encoding algorithm itself.

The comparison between the alignment strategies reveals a distinct mathematical pattern that strongly validates the theoretical expectations discussed in Section XVIII-B3. For the RMSE, Chamfer, and Hausdorff distances, the robust (decoupled) method consistently yields lower error values than the classical (coupled) approach. By deriving the rigid transformation solely from the cleaned point cloud (Q_{clean}), the robust alignment prevents floating noise from skewing the global pose, ensuring a much tighter geometric fit for the structural core of the subject.

The notable exception to this pattern is the Mean Error, which scores slightly higher under the robust method, particularly at the lowest bitrate of 1 Mbps. This behaviour is mathematically consistent with the decoupled logic: because the robust alignment perfectly anchors the main body without compromising its pose to "reach" extreme outliers, those unmitigated outlying points remain at a greater absolute distance from the reference, thereby slightly inflating the linear average of the distances. Conversely, the classical method artificially pulls the reference toward these outliers; while this slightly reduces the linear Mean Error, it severely penalises the squared (RMSE) and worst-case (Hausdorff) distances. Ultimately, the robust alignment correctly exposes the true magnitude of these extreme errors rather than masking them, providing a much more reliable evaluation of the system's actual performance limits.

C. Point-wise Analysis (Landmarks)

The global metrics quantify *how much* the cloud is degraded, but not *where*. To localise the error, the analysis is repeated on six anatomical landmarks—face, torso, left and right arm, left and right leg—chosen to sample regions with different geometric and motion characteristics. The same six landmarks are reused for the 2D analysis.

After the reconstructed cloud has been aligned (Section XVIII-B3), each metric below is evaluated locally at every landmark.

a) Local 3D error.: For a landmark ℓ taken on the reference, its reconstruction error is the Euclidean distance to the nearest reconstructed point, retrieved through a KD-tree nearest-neighbour search:

$$e_{\text{3D}}(\ell) = \min_{q \in Q} \|\ell - q\|_2. \tag{24}$$

The KD-tree makes this nearest-neighbour query efficient; it is the same nearest-point principle used by the global distance $d(q, P)$, here restricted to the landmark set rather than the whole cloud.

b) Displacement vector.: Beyond its magnitude, the error is decomposed along the three axes $(\Delta x, \Delta y, \Delta z)$, indicating the direction in which the reconstructed point is displaced with respect to the reference landmark.

c) Local density.: The point density in a neighbourhood of the landmark is measured on the original and on the reconstructed cloud, and their variation is reported, to reveal whether the region is locally thinned or thickened by the reconstruction.

As an example, Table V reports the point-wise 3D study for a single landmark—the face—at 10 Mbps: the local 3D error and its directional components are compared between the classical and the robust alignment, across the encoder, pre-erosion and post-erosion stages. The other landmarks follow the same scheme.

	Classical Method (Coupled)			**Robust Method (Decoupled)**		
(lr)2-4 (lr)5-7 **Metric (Voxel)**	**Encoder**	**Pre-Eros.**	**Post-Eros.**	**Encoder**	**Pre-Eros.**	**Post-Eros.**
Mean 3D Error	1.14×10^{-13}	1.871	1.774	1.14×10^{-13}	2.056	2.100
Directional Error Components						
Direction X	0	0.070	−0.021	0	0.087	0.177
Direction Y	1.14×10^{-13}	−1.385	−1.086	-1.14×10^{-13}	−1.307	−1.301
Direction Z	0	1.256	1.403	0	1.585	1.639
Local Density Analysis						
Original	1	1	1	1	1	1
Reconstructed	1	1	1	1	1	1
Variation	0	0	0	0	0	0

TABLE V

POINT-WISE 3D RESULTS FOR THE *face* LANDMARK AT 10 MBPS (VOXEL UNITS). THE TABLE COMPARES THE MEAN LOCAL ERROR AND ITS DIRECTIONAL DECOMPOSITION BETWEEN THE CLASSICAL AND THE ROBUST ALIGNMENT, ACROSS THE ENCODER, PRE-EROSION AND POST-EROSION STAGES.

Aggregating the local 3D error over the six landmarks summarises how the point-wise distortion evolves with the rate. Figure 11 reports this mean error for the two alignment modes, in voxels and millimetres.

D. 2D Projection Distance

At the third and final level, geometric fidelity is assessed in the 2D domain. While the 3D metrics capture absolute spatial deformation, the 2D projection distance measures the displacement between the screen-space projections of corresponding points versus rate, directly reflecting the distortion perceived by the client upon rendering. The analysis uses the same six landmarks introduced for the point-wise 3D study.

1) Analysis Metrics: Each 3D landmark is projected onto the face of the cube on which it falls, giving a pair of normalised coordinates $(U, V) \in [0, 1]$ on the projection surface. Two families of metrics are then computed.

a) Pixel displacement.: The position of a landmark on the image is compared between the original and the reconstructed point through the pixel displacement

$$\text{Error}_{2D} = \sqrt{(U_{\text{orig}} - U_{\text{rec}})^2 + (V_{\text{orig}} - V_{\text{rec}})^2}, \qquad (25)$$

i.e. how far the landmark moves on the projected image.

b) Band errors.: At the landmark pixel, three image bands are compared between original and reconstructed maps: *geometry* (depth level), *occupancy* (mask) and *texture* (colour). Each band yields its own error versus bitrate.

	Reconstructed pixel			**Original**
(lr)2-4 **Metric**	**Encoder**	**Pre-Eros.**	**Post-Eros.**	**(fixed)**
Pixel (U, V)	$(520, 412)$	$(536, 420)$	$(511, 402)$	$(520, 412)$
Pixel error (px)	0	1	1	—
Band errors				
Geometry (0–255)	9	1	11	9
Occupancy (mask)	0	6	15	0
Texture (RGB)	30.69	2.00	46.10	30.69

TABLE VI

POINT-WISE 2D RESULTS FOR THE *face* LANDMARK AT 10 MBPS, PROJECTED ON THE TOP CUBE FACE. THE RECONSTRUCTED-PIXEL READING IS SHOWN ACROSS THE THREE RECONSTRUCTION STAGES; THE ORIGINAL-PIXEL READING IS THE FIXED REFERENCE. AT THIS BITRATE THE COUPLED AND DECOUPLED MODES COINCIDE.

c) Two readings.: The band errors can be read in two ways. Reading the pixel at the projection of the *reconstructed* point yields a value that moves from one bitrate to another, so the curve mixes two effects—the image degrading and the reading point jumping—and is therefore noisy. Reading instead at the projection of the *original* landmark, which is fixed, isolates the image quality at a reference point and yields a clean rate–distortion curve that depends on the bitrate alone.

2) Coupled vs. Decoupled Alignment: The reconstructed-pixel reading inherits the quality of the initial 3D alignment, and it is worth making explicit how an alignment error propagates all the way to the image. The pixel of a reconstructed landmark is obtained by projecting its 3D position onto the cube face; this projection is computed in the *aligned* frame, i.e. after the rigid transformation $(R, \mathbf{t})$ estimated by ICP has been applied. If that transformation is biased—because a few outliers pulled the registration away from the correct pose—the whole reconstructed cloud is rotated or translated by a small spurious amount. Every landmark is then projected from a slightly wrong 3D position, and the resulting (U, V) coordinates are shifted accordingly. The pixel displacement therefore no longer reflects only the coding distortion of the map: it also absorbs a rigid offset that has nothing to do with how well the image was encoded.

This is precisely the effect the two alignment strategies let us separate. Under the *coupled* (classical) mode, the same outliers that bias the 3D pose also bias every 2D projection, inflating the pixel error with a component that is constant across the image. Under the *decoupled* (robust) mode, the pose is estimated on the outlier-filtered cloud, so the projections start from a correct 3D position and the residual 2D error reflects the true coding distortion much more faithfully. The reconstructed-pixel results are accordingly reported for both modes. The original-pixel reading, being taken at a fixed reference position, is unaffected by the alignment and is reported once.

The following results make this difference visible, and motivate adopting the robust alignment as the reference for the reconstructed-pixel analysis.

a) Why the robust alignment is preferred.: The plots confirm the expected behaviour: the coupled mode carries a larger, roughly constant pixel offset, while the decoupled

mode brings the reconstructed-pixel error close to the clean original-pixel curve. By estimating the pose on the outlier-filtered cloud, the robust alignment removes the spurious rigid shift and exposes the true coding distortion. It is therefore adopted as the reference for all reconstructed-pixel results.

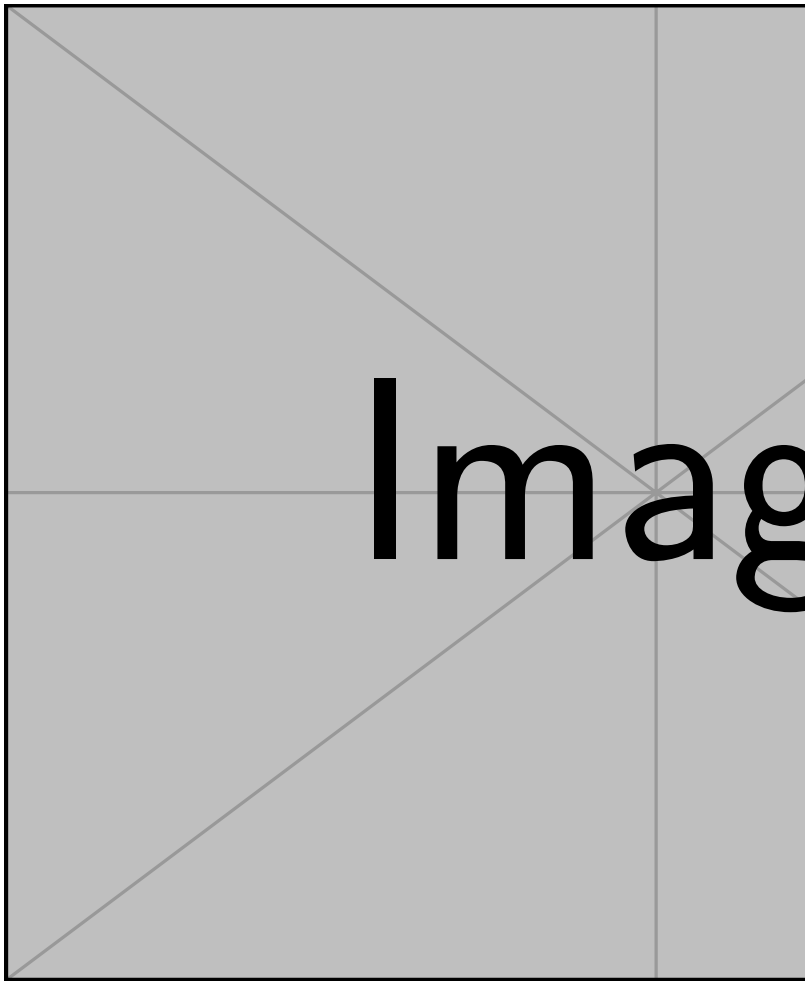


Fig. 5. The final 2D Atlas (Super-Frame) ready for video encoding. The image illustrates the horizontal concatenation of the Geometry, Texture, and Occupancy channels, each organized in the fixed 3×4 cross-shaped layout to preserve temporal coherence.

[Image: BB CENTRO? QUALCOSA DI SIMILE ?]

Fig. 6. The *loot* subject inside the native voxel cube.

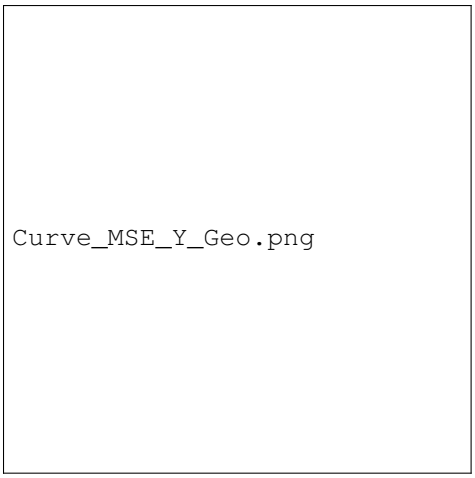


Fig. 7. Depth-map coding error: MSE versus bitrate.

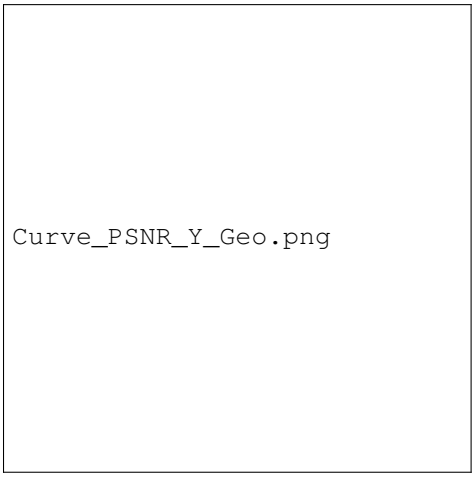


Fig. 8. Depth-map coding error: PSNR versus bitrate.

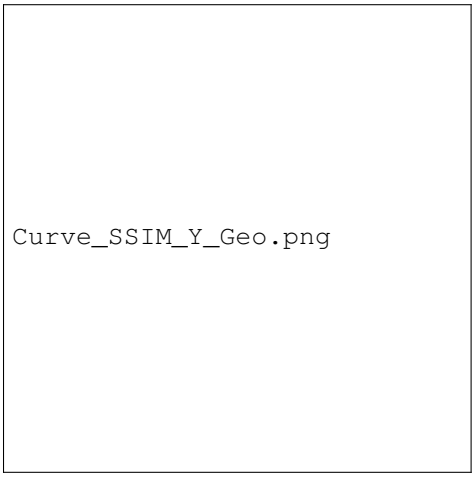


Fig. 9. Depth-map coding error: SSIM (luma channel) versus bitrate.

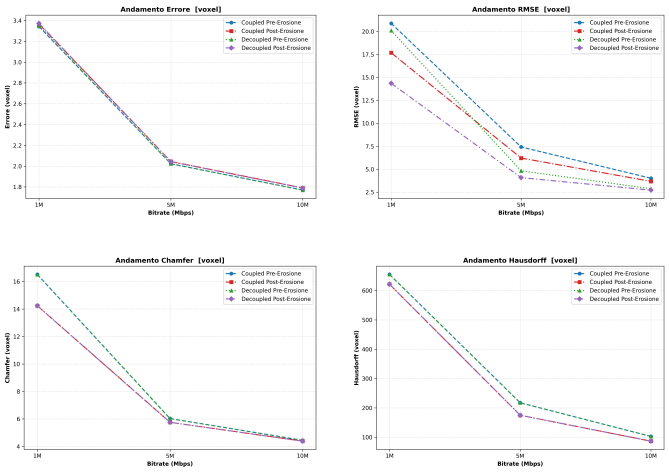


Fig. 10. 3D geometric error metrics versus bitrate. Top left: Mean Error. Top right: RMSE. Bottom left: Chamfer distance. Bottom right: Hausdorff distance. The plots contrast the classical (coupled) and robust (decoupled) alignment strategies across the tested configurations.

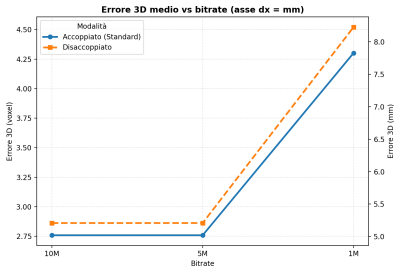


Fig. 11. Mean local 3D error over the six landmarks versus bitrate, for the classical (coupled) and robust (decoupled) alignment modes (left axis: voxel; right axis: mm).

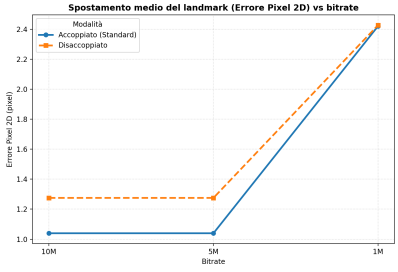


Fig. 12. Mean pixel displacement of the landmarks versus bitrate, for the coupled and decoupled alignment modes.